

4. Exercise: TSP

Abstract

The Traveling Salesman Problem (TSP) is a widely studied combinatorial optimization challenge, aiming to find the shortest route visiting a set of cities exactly once. Formally defined for a complete, weighted graph, the TSP is NP-hard, leading to heuristic solutions for practical instances. This study implements and compares two heuristics, namely the 2-Opt and nearest-neighbor methods. Experimental evaluation involves assessing the algorithms based on execution time and the number of swaps performed, shedding light on their comparative performance.

1 Introduction

The TSP (Traveling Salesman Problem) is a well-known and well-researched combinatorial optimization problem.

Figuratively speaking, the challenge is to determine the shortest possible route that visits each city of a set of cities exactly once.

It is formally defined as follows: Given a Graph $G(V, E)$ that is complete and weighted, find the round tour visiting every node exactly once that has the minimal possible cost (is the shortest).

The Traveling Salesman Problem is an NP-hard problem, so there were different heuristic and approximative algorithms developed to solve the problem in practice on larger instances. Two of them (2-Opt heuristic) and (nearest-neighbor heuristic).

We will implement both to compare them in terms of running time and quality (ratio of found route length to optimal route length).

2 Preliminaries

2.1 Nearest-neighbor heuristic

The nearest-neighbor heuristic is a simple heuristic algorithm that solves the TSP approximately. The main idea is to construct a tour by iteratively selecting the nearest unvisited city at each step, starting from an initial city, until all cities have been visited.

Start: Choose (arbitrarily/first) starting city to start the tour
Iteration: At each step, select the nearest unvisited city to the current city in terms of distance. Move to the selected city and mark it as visited.
Termination: Repeat the iteration until all cities have been visited. Once all cities are visited, return to the starting city to complete the tour.

Time complexity:

Naive approach: Number of elements * Time to find minimum edge = $O(n^3)$.

Using priority queue (by creating a priority queue of size $O(n^2)$ to save all edges) = $O(n^2 * \log(n))$.

2.2 2-opt heuristic

The 2-opt heuristic is a local search algorithm commonly used to improve solutions in the TSP.

In contrast to construction heuristics like the nearest-neighbor heuristic described in section 2.1, the 2-opt heuristic starts with arbitrary feasible solution and refines this iteratively. It does that by swapping pairs of edges to reduce the total tour length. The intuition behind the 2-opt heuristic is to explore local changes in the tour that might lead to a shorter overall distance, eventually converging to a more optimal solution.

Initial Tour: Start with an initial tour, which could be obtained using a construction heuristic like the nearest-neighbor algorithm.
Iterative Improvement: Consider all possible pairs of edges in the current tour (excluding adjacent edges, as

Algorithm 1: Nearest-Neighbor Heuristic for TSP

Data: Set of cities $C = \{c_1, c_2, \dots, c_n\}$,
Distance matrix $D = [d_{ij}]$
Result: Approximate solution P with
minimized tour cost

```
1 Initialize an empty tour  $P$ 
2 Select a starting city  $c_{\text{start}}$  arbitrarily
3 Mark  $c_{\text{start}}$  as visited
4 Add  $c_{\text{start}}$  to  $P$ 
5 while There are unvisited cities do
6   Find the nearest unvisited city  $c_{\text{next}}$  to the
   last visited city in terms of distance
7   Mark  $c_{\text{next}}$  as visited
8   Add  $c_{\text{next}}$  to  $P$ 
9 Complete the tour by returning to the starting
  city
10 return  $P$ 
```

swapping them would not change the tour length).
Edge Swapping: For each pair of edges, evaluate the potential improvement in tour length by swapping them. Best Improvement: If swapping a pair of edges results in a shorter tour, perform the swap. Termination: Repeat the process until no further improvements can be made.

Time complexity = $O\left(\frac{1}{\phi} \cdot n^6 \cdot \log n\right)$ Storandt [2023]. Here ϕ is the amount of perturbation we add while doing the smoothed analysis. Decreasing the value of ϕ that is decreasing the perturbation makes the time complexity near to the worst-case, that is exponential time complexity, and increasing the perturbation makes the time complexity closer to the average case time complexity.

The time complexity for searching a pair of edges that will reduce the total weight: $O(n^2)$ as a simple iterative search strategy is followed by considering all possible pairs of edges in the given path to reduce the path length. As the number of edges in one path = $n - 1$, the total number possible pairs of edges = $\frac{n*(n-1)}{2}$.

Algorithm 2: 2-opt Heuristic for TSP

Data: Initial tour P , Distance matrix
 $D = [d_{ij}]$

Result: Improved tour P_{opt}

```
1 repeat
2    $\text{improvement} \leftarrow \text{false}$ 
3   for  $i \leftarrow 1$  to  $\text{length}(P) - 2$  do
4     for  $j \leftarrow i + 2$  to  $\text{length}(P)$  do
5       if  $d_{P[i], P[i+1]} + d_{P[j], P[j+1]} >$ 
6          $d_{P[i], P[j]} + d_{P[i+1], P[j+1]}$  then
7           Reverse the sub-tour between
             $P[i + 1]$  and  $P[j]$ 
             $\text{improvement} \leftarrow \text{true}$ 
8 until No improvement is made
```

3 Experimental Evaluation

3.1 Data and Hardware

3.1.1 Dataset

We use 27 TSP instances provided by the math department of the University of Waterloo.¹ The instances are in the TSPLIB format and list between 29 and 71,009 cities (nodes) with their respective position. For each instance, a near-optimal route is provided as well.²

3.1.2 Hardware

All algorithms were evaluated on a machine with an Apple M2 CPU with 3.49 GHz and a memory of 32GB.

3.2 Results

The algorithmic running time results are shown in Table 1, which clearly shows that the 2-opt algorithm has a faster running time than the nearest neighbor algorithm for all cases which uses a naive approach with a time complexity of $O(n^3)$.

¹<https://www.math.uwaterloo.ca/tsp/world/countries.html>

²each < 0.1% divergence from optimal

The final path length comparison is shown in Table 2, which shows that out of 27 cases the 2 opt with nearest neighbor solution gave the shortest route in 24 cases, the 2 opt algorithm with default initial path gave the shortest route in 3 cases and the nearest neighbor algorithm in none. Also, for the cases, the 2 opt algorithm gave a shorter total route length when compared to the nearest neighbor algorithm. Table 3 shows the number of swaps required in the 2-opt algorithm for the case given with sequential nodes as the initial path and the case with the output of the nearest neighbor algorithm is given as the initial path. Here, in all the cases, the 2 opt algorithm with the initial path as the near neighbor algorithm has significantly less number of swaps.

Also, figure 1 to 15 shows 5 types of paths for 3 countries: Djibouti, Luxembourg, and Qatar:

- A) Showing the output of the nearest neighbor algorithm.
- B) Showing the initial path of the 2 opt algorithm.
- C) Showing the state of the 2-opt algorithm in the approximate middle of the number of iterations.
- D) Showing the final output of the 2-opt algorithm.
- E) Showing the final output of the 2-opt algorithm when the nearest neighbor output is given as the initial path for the 2-opt algorithm

4 Discussion and Conclusion

The run time of the 2-opt algorithm is better than the nearest neighbor algorithm, hence the experimental time complexity of the 2-opt algorithm comes to be $= O(n^3)$.

Also, the 2-opt algorithm gives a better path than the nearest neighbor algorithm in all cases, which makes it a better heuristic for the traveling salesman problem. Also, in most cases the 2-opt algorithm when given the nearest neighbor's algorithm as the initial path performs better which is as per our expectation, but in a few cases where it performs worse. Since 2-opt is also an approximation algorithm this is possible that the nearest neighbor algorithm already gives an output with no cuts, but the 2-opt algorithm when

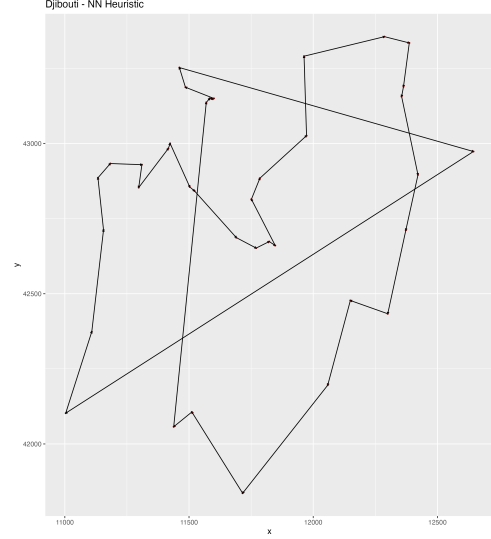


Figure 1: Nearest neighbor output for Djibouti

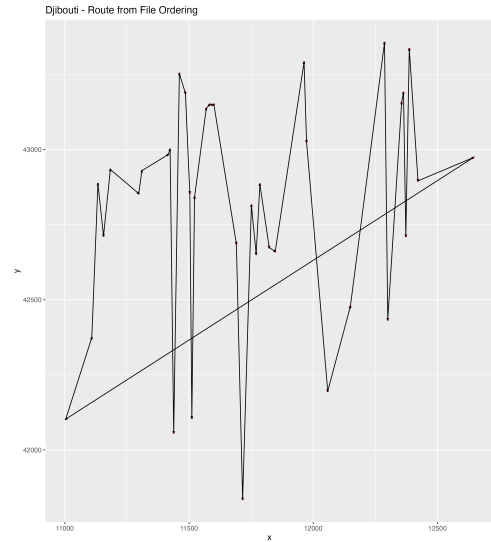


Figure 2: Initial input for 2-opt algorithm for Djibouti

	Country	NN Time	2-Opt Initial Time	2-Opt NN Time
1	Argentina	1832	1024	630
2	Burma	26965	12570	12409
3	China	141432	68428	45355
4	Djibouti	0	0	0
5	Egypt	1156	609	550
6	Finland	2225	1115	866
7	Greece	1945	952	1786
8	Honduras	4892	2369	1831
9	Ireland	1370	745	512
10	Japan	1919	1048	838
11	Kazakhstan	2144	978	861
12	Luxembourg	11	6	7
13	Morocco	4123	2214	1970
14	Nicaragua	217	124	106
15	Oman	53	47	42
16	Panama	1346	725	496
17	Qatar	0	0	0
18	Rwanda	38	24	14
19	Sweden	13613	8911	5442
20	Tanzania	681	366	361
21	Uruguay	6	5	3
22	Vietnam	11123	6203	5629
23	Western_Sahara	1	0	0
24	Yemen	1146	576	504
25	Zimbabwe	9	6	5
26	Canada	378	215	164
27	Italy	6199	3119	2799

Table 1: Algorithm Running Times in ms

starting from the beginning finds a better way. But overall the performance of the 2-opt algorithm when the nearest neighbor algorithm's output is given as the initial path is significantly better than giving the points in sequential order as the initial path.

The number of swaps is lower when we give the nearest neighbor algorithm's output as the initial path as it is already a good solution as compared to a random path, hence, the room for improvement is little. Therefore, the 2-opt algorithm finishes faster.

For the given figures for the countries: Djibouti, Luxembourg, and Qatar, we clearly see that the 2 opt algorithm gives us a polygon covering all points without any cuts. Which shows the 2-opt algorithm is work-

ing well. In the case of the nearest neighbors, all 3 countries have cuts, which clearly shows a better path could have been formed. This happens due to the greedy nature of the algorithm, which exhausts the best possible outcome to greedily choose the nearest neighbor.

References

S. Storandt. Lecture: 8. smoothed analysis - 1, 2023.

	Country	Optimal Length	NN Path Length	2-Opt Initial Path Length	2-Opt NN Path Length
1	Argentina	837479	1065260	945761	921129
2	Burma	959011	1207410	1081290	1041620
3	China	4565452	5654790	5181260	4934990
4	Djibouti	6656	9748	6710	7245
5	Egypt	172386	218143	193727	187917
6	Finland	520527	653781	588846	563275
7	Greece	300899	388741	337688	332078
8	Honduras	177092	217793	199393	190498
9	Ireland	206171	264414	232741	224734
10	Japan	491924	618826	554851	535417
11	Kazakhstan	1061881	1349320	1201710	1164520
12	Luxembourg	11340	15285	12734	12368
13	Morocco	427377	534347	483106	461633
14	Nicaragua	96132	120895	106697	104941
15	Oman	86891	110558	94301	95634
16	Panama	114855	148384	128348	124983
17	Qatar	9352	11892	10453	9888
18	Rwanda	26051	33710	29611	28538
19	Sweden	855597	1078230	967731	930292
20	Tanzania	394718	497280	444597	429606
21	Uruguay	79114	100920	88326	85530
22	Vietnam	569288	723592	636037	621207
23	Western_Sahara	27603	36388	27601	27748
24	Yemen	238314	300898	265854	258139
25	Zimbabwe	95345	116863	105651	102758
26	Canada	1290319	1650190	1475540	1418960
27	Italy	557315	704260	628860	603729

Table 2: Result Path Length

	Country	2-Opt Initial Swaps	2-Opt NN Swaps
1	Argentina	53128	2087
2	Burma	70086	9689
3	China	278470	19069
4	Djibouti	55	24
5	Egypt	20254	2193
6	Finland	26883	3102
7	Greece	19451	3117
8	Honduras	82850	2005
9	Ireland	28585	2601
10	Japan	22840	2969
11	Kazakhstan	32361	2948
12	Luxembourg	3708	253
13	Morocco	33860	3989
14	Nicaragua	17198	693
15	Oman	4533	604
16	Panama	41894	1362
17	Qatar	306	99
18	Rwanda	6907	271
19	Sweden	54704	7158
20	Tanzania	14667	1828
21	Uruguay	1669	216
22	Vietnam	43715	6558
23	Western_Sahara	34	15
24	Yemen	20765	2218
25	Zimbabwe	2017	243
26	Canada	14098	1517
27	Italy	40070	5031

Table 3: 2-Opt Swaps

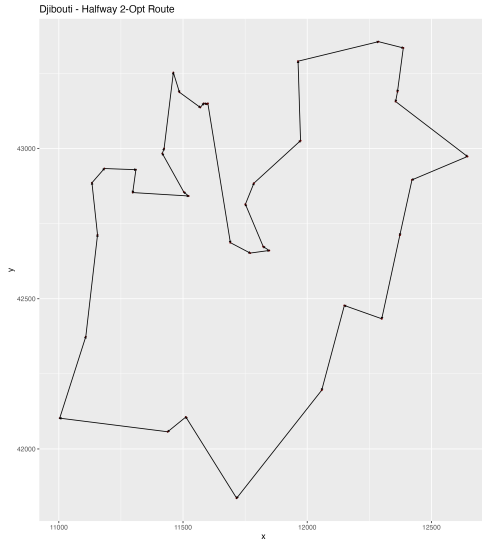


Figure 3: Middle position for 2-opt algorithm for Djibouti



Figure 5: Final output for 2-opt algorithm for Djibouti with nearest neighbor output as initial path.



Figure 4: Final output for 2-opt algorithm for Djibouti

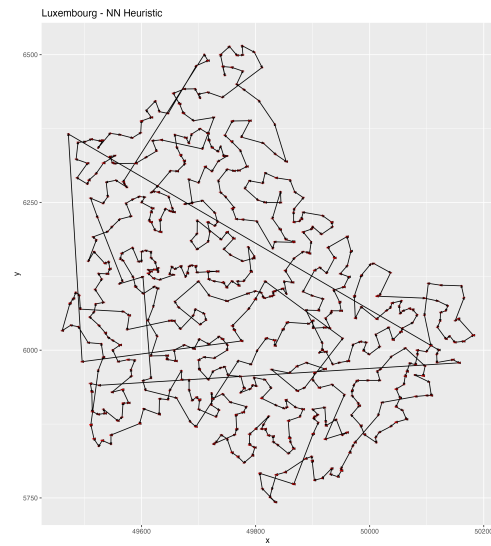


Figure 6: Nearest neighbor output for Luxembourg

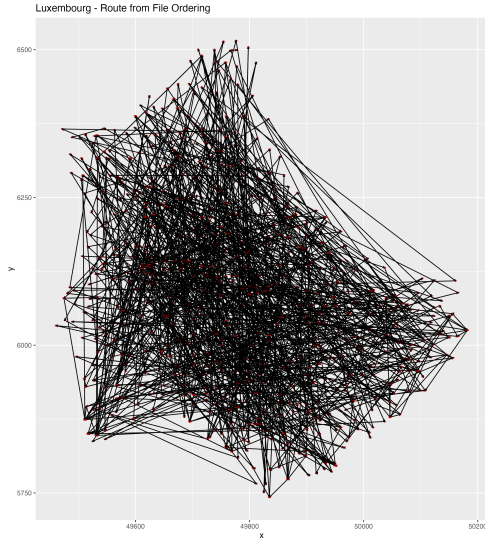


Figure 7: Initial input for 2-opt algorithm for Luxembourg



Figure 9: Final output for 2-opt algorithm for Luxembourg

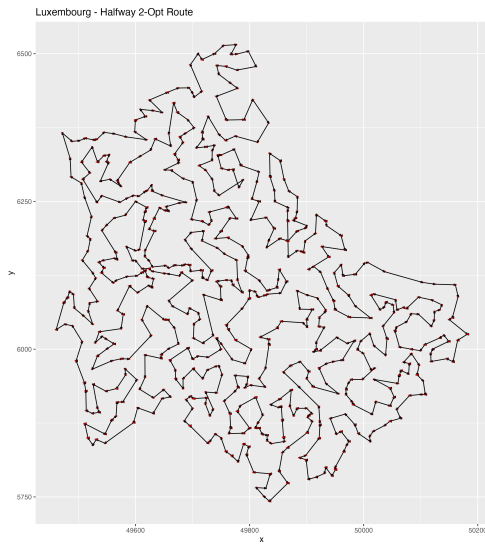


Figure 8: Middle position for 2-opt algorithm for Luxembourg



Figure 10: Final output for 2-opt algorithm for Luxembourg with nearest neighbor output as initial path.

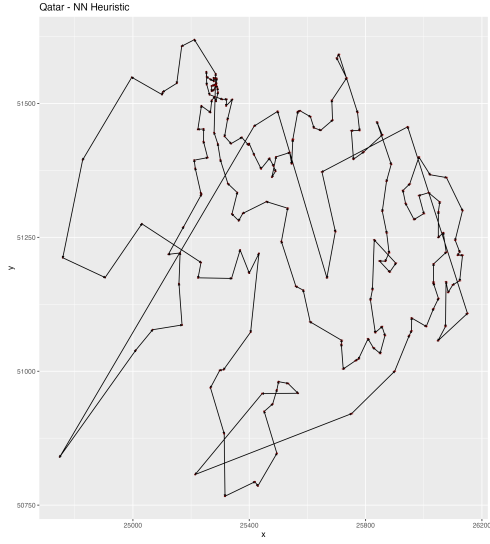


Figure 11: Nearest neighbor output for Qatar



Figure 13: Middle position for 2-opt algorithm for Qatar

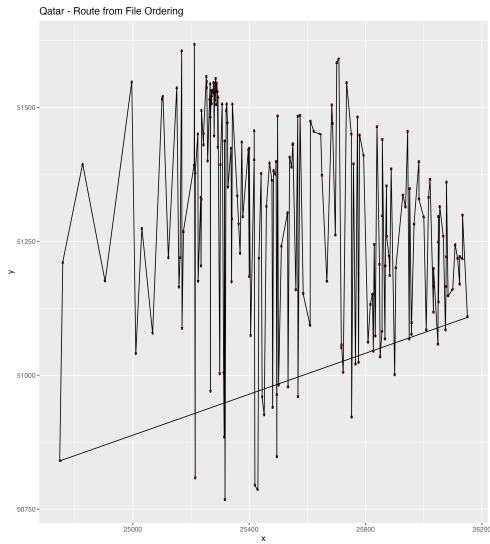


Figure 12: Initial input for 2-opt algorithm for Qatar

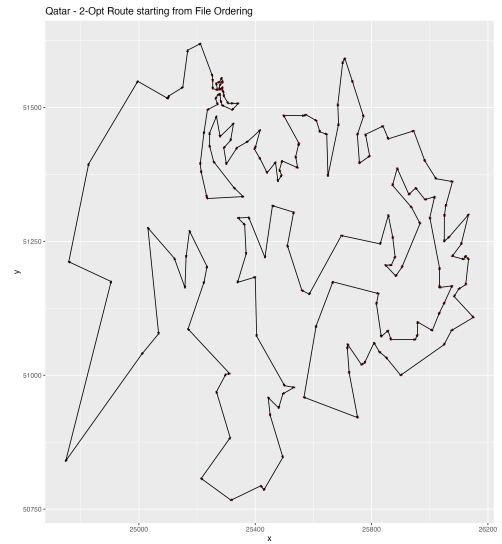


Figure 14: Final output for 2-opt algorithm for Qatar

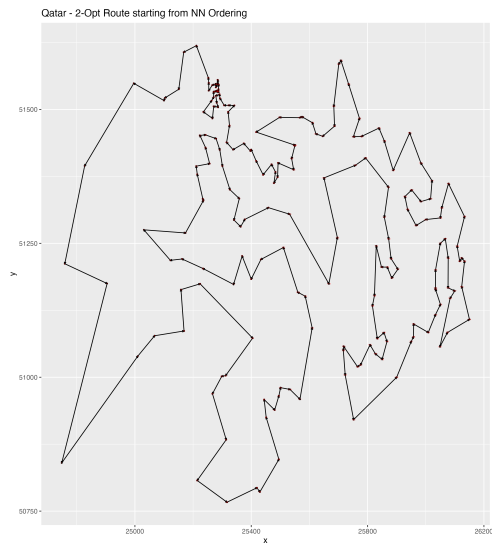


Figure 15: Final output for 2-opt algorithm for Qatar with nearest neighbor output as the initial path.